

# Scala Ecosystem Security Model

## Objective

Build a lightweight model of the Scala ecosystem that helps identify **weak links where security or maintenance intervention would protect disproportionate ecosystem value.**

The core question is:

**Which high-value software components depend on components with weak maintenance health or weak security health?**

## Model

The ecosystem is represented as a **graph of software components and their dependencies.**

Each component receives three normalized scores:

- **Value** — how important the component is to the ecosystem.
- **Maintenance Health** — how actively and sustainably the component is maintained.
- **Security Health** — how much confidence we have in its current security posture.

The model then evaluates dependency relationships and identifies components with **low Maintenance Health or Security Health that have a large downstream blast radius.**

The intended output is a ranked **intervention map**: the places where funding, maintenance support, auditing, or other security work could protect the greatest amount of ecosystem value.

## Initial Scoring Approach

For the first version, the scoring mechanism can reuse the general approach of the **OpenSSF Criticality Score**, adapted independently for the three dimensions.

The exact scoring formula is secondary to selecting meaningful inputs.

Possible initial signals include:

- **Value**: direct and transitive dependents, downloads, stars/forks, ecosystem adoption, adoption by the industry.
- **Maintenance Health**: recent activity, active contributors, contributor concentration, release cadence, issue/PR responsiveness.
- **Security Health**: OpenSSF Scorecard signals, vulnerability-reporting practices, previous audits/security reviews, and other observable security work.

The choice and weighting of these inputs should be refined with ecosystem and security expertise.

## Dependency Model

The model should go beyond package dependencies where practical. Relevant relationships may include:

- runtime and library dependencies;
- build and development tooling;
- CI, release, and publishing infrastructure.

The initial implementation can deliberately use a coarse subset and become more complete through iteration.

## Near-Term Goal

Produce a small, coarse model of a representative Scala ecosystem subset, inspect the weak links it identifies, and iteratively refine the inputs and dependency discovery.

The purpose is to turn ecosystem security from a broad concern into **concrete, comparable intervention priorities** that can support the design and funding of a wider Scala ecosystem security program.